

Generalized Backpropagation as Signal Flow

Paul Alexander Bilokon

August 9, 2026

Abstract

Algorithmic differentiation (AD) is widely used in scientific computing and machine learning, yet its mathematical structure and semantic foundations remain fragmented across numerical analysis, automatic differentiation systems, and deep-learning engineering. This paper develops a unified foundation for AD by combining: (i) the categorical structure of differential categories and adjoint linearisation; (ii) the denotational semantics of programs via Scott domains and continuous maps; and (iii) the computational structure of neural networks as compositional morphisms in a Cartesian closed category enriched with a differential combinator. We show that reverse-mode AD arises as the functorial pullback of covectors along morphisms in an appropriate co-Kleisli differential setting, and that the backpropagation algorithm of Rumelhart–Hinton–Williams is precisely the instantiation of this adjoint pullback on computational graphs. This yields a conceptual and mathematical explanation of why backpropagation works, how it generalises to differentiable programming languages, and how higher-order derivatives, implicit differentiation, and differentiable fixed-point computations arise naturally from differential structure in domain-theoretic models. We further show how the semantics of neural networks, meta-learning procedures, and implicit layers can be captured in this framework.

The integrated development then treats exact adjoint pullback as the canonical zero-defect member of a larger family of backward signal-flow procedures. In the linear-additive case the backward field satisfies $s = q + \mathcal{B}s$; on a feed-forward DAG exact backpropagation is the finite Neumann solution and reverse traversal is a triangular solve. Alternative derivatives, random or learned transports, shortcut feedback, local semantic losses, numerical solvers, and finite credit horizons become comparable propagation calculi. We prove descent conditions for inexact directions, define multiple notions of propagation optimality, and test them on propositional logic and MNIST. The experiments distinguish gradient fidelity from learning ability, identify budget-dependent optimal credit horizons, expose the importance of temporal credit coverage in sequential networks, and motivate a coverage-constrained adaptive sparse propagator.

Keywords: algorithmic differentiation; backpropagation; Cartesian differential categories; domain theory; adjoint pullback; credit assignment; signal flow; propagation optimality; propositional logic; MNIST.

1 Introduction

Algorithmic differentiation (AD) is a key component of modern numerical computation. In deep learning, reverse-mode AD appears concretely as the backpropagation algorithm, enabling efficient gradient computation in neural networks with millions or billions of parameters. Yet despite its central role, the theoretical understanding of AD remains split across several communities:

- numerical analysts formalise AD via Jacobian accumulation and operator overloading [Griewank and Walther, 2008, Baydin et al., 2018];
- machine learning practitioners view backpropagation as a computational recipe tied to network architecture [Rumelhart et al., 1986];

- category theorists develop abstract axioms for differentiation in Cartesian differential categories and tangent categories [Blute et al., 2009, Cockett and Cruttwell, 2014]; and
- denotational semanticists analyse programs as continuous maps over complete partial orders, often separately from differentiability [Abramsky and Jung, 1994].

This paper provides a unified foundation that spans these viewpoints and then extends the exact theory into a quantitative theory of generalised backward information flow.

Contributions of the exact semantic development

The original development makes the following contributions:

- (i) It constructs a denotational semantics of differentiable programs using Scott domains, with continuous linearisations in a cpo-enriched setting.
- (ii) It develops a categorical framework in which algorithmic differentiation is governed by a differential combinator D , and reverse mode is an adjoint pullback when suitable reverse/dual structure exists.
- (iii) It identifies neural-network backpropagation with composition of adjoint linearisations in a co-Kleisli differential setting.
- (iv) It analyses forward-mode, reverse-mode, and higher-order AD and relates Hessian-vector products to iterated differentiation.
- (v) It gives a fixed-point account of implicit differentiation, deep equilibrium-style models, and neural ODEs.
- (vi) It applies the theory to neural networks, meta-learning, differentiable programming, scientific adjoints, and implicit optimisation layers.

The code behind this paper is available at https://github.com/thalesians/generalized_backprop

2 Preliminaries

We recall basic notions underlying algorithmic differentiation, computational graphs, and denotational models of programs. We assume familiarity with categories, domains, and elementary real analysis.

2.1 Computational graphs

A computational graph is a finite directed acyclic graph (DAG)

$$G = (V, E),$$

whose vertices $v \in V$ represent primitive operations (addition, multiplication, activation functions, control-flow primitives, or externally defined functions) and whose edges represent data dependencies. Each node v is annotated with an operation

$$\text{op}_v : \mathbb{R}^{k_v} \rightarrow \mathbb{R}^{m_v}.$$

A program or neural network defines such a graph together with designated input and output nodes. Given inputs $x \in \mathbb{R}^n$, evaluation produces values on all nodes by topological order. We view this as computing a function

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m.$$

2.2 Differentiable functions and Jacobians

For $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ differentiable at x , the Jacobian is

$$J_f(x) = \left(\frac{\partial f_i}{\partial x_j}(x) \right)_{i,j}.$$

Forward-mode AD computes $J_f(x)v$; reverse-mode computes $J_f(x)^\top w$.

2.3 Forward and reverse AD

Forward-mode AD interprets each computation in the graph on dual numbers $x + \varepsilon v$, propagating directional derivatives alongside values. Reverse-mode AD performs an adjoint sweep: starting with a covector w at the output, it propagates sensitivities backward using the chain rule. This computes $J_f(x)^\top w$ with cost proportional to one graph traversal in the usual reverse-accumulation model [Griewank and Walther, 2008, Baydin et al., 2018].

2.4 Neural networks as compositional maps

A feed-forward network with layers

$$f = f_L \circ \cdots \circ f_1$$

is a composite of smooth functions. The chain rule yields

$$Df(x) = Df_L(z_{L-1}) \circ \cdots \circ Df_1(x),$$

where z_k is the k th layer output. Neural-network backpropagation corresponds to propagating adjoints through this composition. The purpose of the later sections is to express this fact categorically and domain-theoretically.

2.5 Domains and continuous functions

We use standard domain-theoretic notions [Abramsky and Jung, 1994]. A domain (cpo) is a partially ordered set $(D, \sqsubseteq)$ such that every directed subset has a least upper bound, together here with a least element $\perp$. A function $f : D \rightarrow E$ between domains is Scott-continuous if it preserves suprema of directed sets:

$$f\left(\bigsqcup_i x_i\right) = \bigsqcup_i f(x_i).$$

Continuous functions model computable programs; least fixed points of continuous self-maps model recursion.

2.6 Notation

We write $Df(x)$ for the differential at x and $Df(x)^*$ for its adjoint when a suitable dual pairing is present. We write $\mathcal{C}$ for a Cartesian closed category, **CPO** for a category of domains and continuous maps, and $\langle x, v \rangle$ for a pair in a product object.

3 Categorical Foundations

We recall the categorical structures required to model differentiable programs and algorithmic differentiation.

3.1 Cartesian closed categories

A category $\mathcal{C}$ is Cartesian closed if it has a terminal object 1 , binary products $A \times B$ with projections π_1, π_2 , and exponentials B^A with evaluation $\text{ev} : B^A \times A \rightarrow B$. Programs in simply typed λ -calculus admit interpretation in any CCC. Neural networks correspond to morphisms

$$f : X \longrightarrow Y$$

built from composition, products, and primitive nonlinear maps.

3.2 Differential combinator

A Cartesian differential category (CDC) [Blute et al., 2009] is a Cartesian left-additive category equipped with a differential combinator

$$D : \mathcal{C}(A, B) \longrightarrow \mathcal{C}(A \times A, B),$$

assigning to each morphism $f : A \rightarrow B$ a morphism $Df : A \times A \rightarrow B$ interpreted as its directional derivative. In the notation used in the original manuscript, the relevant axioms include linearity in the second argument, regularity $Df(a, 0) = 0$, the chain rule

$$D(g \circ f) = Dg \circ \langle f \circ \pi_1, Df \rangle,$$

additivity, and compatibility with projections. These axioms abstract the familiar derivative laws.

3.3 Tangent and reverse structures

A CDC admits a tangent functor

$$T(A) = A \times A, \quad T(f) = \langle f \circ \pi_1, Df \rangle.$$

When the relevant linear maps carry coherent dual/dagger structure, one can dualise linearisation and obtain a cotangent-style reverse action. Reverse derivative categories axiomatise this additional reverse structure directly [Cockett et al., 2020].

Remark 3.1 (Reverse differentiation is additional structure). The forward differential combinator of a CDC does not by itself provide a canonical adjoint for every object and map. Every statement below involving $Df(x)^*$ is therefore read under the stated assumption that the required dual pairings or reverse-derivative structure exist. This makes precise the original manuscript’s qualifier “when it exists.”

3.4 Coalgebraic structure and co-Kleisli categories

Let $(!, \delta, \varepsilon)$ be a comonad on $\mathcal{C}$ suitable for modelling controlled duplication, as in differential linear logic. The co-Kleisli category $\mathcal{C}_!$ has the same objects as $\mathcal{C}$ and morphisms $A \rightarrow B$ represented by maps $!A \rightarrow B$ in $\mathcal{C}$. When the differential combinator is compatible with the comonad, the co-Kleisli category inherits differentiation [Blute et al., 2006, 2020]. Composition in $\mathcal{C}_!$ models composition of differentiable programs.

3.5 Interpretation of programs

A program in a differentiable functional language is interpreted as a morphism in $\mathcal{C}_!$. Primitive operations are morphisms; control structures use the CCC structure; differentiation applies the differential combinator to program denotations. In this sense, differentiable programming internalises differentiation in the semantic model [Abadi and Plotkin, 2020].

3.6 Functoriality of AD

The chain-rule identity

$$D(g \circ f) = Dg \circ \langle f \circ \pi_1, Df \rangle$$

expresses the compositionality at the heart of AD: differentiation of a composite program is computed compositionally from derivatives of its parts.

4 Domain-Theoretic Semantics of Differentiable Programs

We now introduce the denotational setting used in the original manuscript. The goal is to combine recursive program semantics with a linearisation operation.

4.1 Domains and continuous maps

A pointed cpo $(D, \sqsubseteq, \perp)$ admits suprema of directed sets. A Scott-continuous function $f : D \rightarrow E$ preserves such suprema. This is the standard semantic substrate for computable approximation and recursive definitions [Abramsky and Jung, 1994].

4.2 Semantic interpretation of programs

Let **CPO** denote a chosen category of domains and continuous functions. A typed program t is interpreted as a continuous function

$$\llbracket t \rrbracket : D_X \longrightarrow D_Y,$$

with product, sum, and exponential structure supplied where the chosen domain model supports it. Primitive arithmetic operations can be lifted to appropriate real-number domains, and program composition is interpreted compositionally.

4.3 Differentiability in domains

To capture differentiability, the original manuscript considers real-number domains $D_{\mathbb{R}}$ equipped with an order completion and asks for a Scott-continuous linear map

$$Df(x) : D_{\mathbb{R}}^n \longrightarrow D_{\mathbb{R}}^m$$

satisfying a Gâteaux-style limit property

$$\lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon v) - f(x)}{\varepsilon} = Df(x)(v)$$

whenever the limit is meaningful in the chosen enriched domain model.

Remark 4.1 (Modelling assumption). A differential structure is not automatic on an arbitrary category of cpos. The claims in this section should be read as assumptions on a domain model enriched with compatible additive and differential structure. This qualification does not affect the later finite-dimensional experiments, where all maps live in Euclidean spaces.

4.4 Least fixed points and recursion

Programs with recursion or loops are interpreted using least fixed points. For a Scott-continuous endomap $\Phi : D \rightarrow D$ on a pointed cpo, the iterative construction is

$$\text{fix}(\Phi) = \bigsqcup_{n \in \mathbb{N}} \Phi^n(\perp)$$

under the usual hypotheses.

Remark 4.2 (Fixed-point theorem terminology). The original manuscript attributed the displayed iterative formula to Knaster–Tarski. In the pointed-cpo/Scott-continuous setting the standard name for this iterative form is the Kleene fixed-point theorem; Knaster–Tarski gives the corresponding complete-lattice fixed-point result. The semantic role intended by the original text is unchanged.

4.5 Differentiating fixed-point programs

If a fixed point depends differentiably on parameters or inputs, differentiating the fixed-point equation leads schematically to an inverse linear operator of the form

$$(I - D\Phi(z))^{-1}, \quad z = \text{fix}(\Phi),$$

whenever the inverse exists as a continuous linear operator. Section 8 gives the more precise parameterised implicit-differentiation equation.

4.6 Compatibility with categorical structure

The intended semantic model carries both denotational structure (continuity, recursion, fixed points) and differential structure (linearisation and, where available, adjoint maps). The compatibility of these structures is the domain-theoretic foundation on which the subsequent interpretation of AD rests.

5 Algorithmic Differentiation as Linearisation

We now give the semantic and categorical account of AD used in the original paper.

5.1 Linearisation of continuous functions

Definition 5.1 (Domain-theoretic derivative). A continuous map $f : D_X \rightarrow D_Y$ is differentiable at x if there exists a Scott-continuous linear map $L : D_X \rightarrow D_Y$ such that

$$\lim_{\varepsilon \rightarrow 0^+} \frac{f(x + \varepsilon v) - f(x)}{\varepsilon} = L(v)$$

for directions v for which the limit is defined in the chosen domain model. We write $L = Df(x)$.

5.2 The differential combinator as linearisation

For a program t interpreted as $\llbracket t \rrbracket : D_X \rightarrow D_Y$, the differential combinator assigns

$$D(t) : D_X \times D_X \longrightarrow D_Y, \quad D(t)(x, v) = D\llbracket t \rrbracket(x)(v).$$

Proposition 5.2 (Differential combinator equals semantic linearisation). *In a semantic model whose primitive operations and constructors satisfy the CDC axioms and the domain-theoretic differentiability assumptions above, the categorical differential combinator on a program coincides with the semantic linearisation of its denotation.*

Proof. Proceed by structural induction on the program. Primitive numeric operations have their specified continuous derivatives. Product and additive cases follow from the corresponding CDC axioms, while composition follows from the categorical chain rule. Thus the syntactic differential transformation and the denotational linearisation agree by induction. $\square$

5.3 Forward-mode AD

Forward-mode AD is the action of the tangent construction

$$T(f)(x, v) = (f(x), Df(x)v),$$

and compositionality follows from the chain rule:

$$T(g \circ f) = T(g) \circ T(f).$$

5.4 Reverse-mode AD as adjoint linearisation

Given a continuous linear map $L : D_X \rightarrow D_Y$ with adjoint $L^* : D_Y^* \rightarrow D_X^*$ defined by

$$\langle L(v), w \rangle = \langle v, L^*(w) \rangle,$$

reverse mode computes $Df(x)^*w$. For a composite $f = f_L \circ \dots \circ f_1$,

$$Df(x) = Df_L(z_{L-1}) \circ \dots \circ Df_1(x),$$

so

$$Df(x)^* = Df_1(x)^* \circ \dots \circ Df_L(z_{L-1})^*.$$

This is precisely the algebraic shape of backpropagation.

5.5 Functoriality of reverse mode

Theorem 5.3 (Functoriality of reverse-mode AD). *Assume the required adjoints exist coherently. For composable differentiable morphisms f and g , the reverse pullback satisfies the contravariant composition law*

$$R(g \circ f) = R(f) \circ R(g),$$

with base-point dependence supplied by the corresponding forward evaluation.

Proof. Apply the chain rule to $D(g \circ f)$ and then take adjoints. Adjointing reverses composition order, yielding the reverse accumulation law. $\square$

6 Reverse-Mode AD as Adjoint Pullback (Backpropagation)

This section contains the central exact result of the original manuscript.

6.1 Adjoint pullback

Let $f : X \rightarrow Y$ be differentiable and fix $x \in X$. The linearisation is

$$Df(x) : T_x X \rightarrow T_{f(x)} Y.$$

When duals exist, its adjoint is

$$Df(x)^* : T_{f(x)}^* Y \rightarrow T_x^* X.$$

Given an output covector $\bar{y}$, the adjoint pullback computes

$$\bar{x} = Df(x)^*(\bar{y}),$$

which is the abstract sensitivity propagation step.

6.2 Composition of adjoint linearisations

For

$$f = f_L \circ f_{L-1} \circ \dots \circ f_1,$$

the chain rule gives

$$Df(x) = Df_L(z_{L-1}) \circ \dots \circ Df_1(x).$$

Taking adjoints reverses the order:

$$Df(x)^* = Df_1(x)^* \circ Df_2(z_1)^* \circ \dots \circ Df_L(z_{L-1})^*.$$

6.3 Backpropagation on computational graphs

Let G represent a program. Each primitive node v has a primal map $\text{op}_v : X_v \rightarrow Y_v$, derivative $D\text{op}_v(z_v)$, and local adjoint $D\text{op}_v(z_v)^*$. Forward evaluation computes intermediate states in topological order. Reverse-mode AD traverses the graph in reverse topological order, applies the local adjoints, and accumulates all incoming covector contributions.

Theorem 6.1 (Reverse-mode AD = adjoint pullback = backpropagation). *Let $f : X \rightarrow Y$ be denoted by a finite computational graph G , and suppose all required linearisations and adjoints exist. For every output covector $\bar{y}$,*

$$\text{ReverseAD}_G(x; \bar{y}) = Df(x)^*(\bar{y}).$$

Moreover, the classical backpropagation algorithm is exactly this pullback computed operationally by a reverse topological traversal and accumulation of local adjoints.

Proof. View the graph as a composition of primitive morphisms after introducing products to account for fan-out and fan-in. The chain rule composes the corresponding linearisations in forward order. Taking adjoints reverses the order. Reverse topological traversal implements this contravariant composition, while summing adjoint contributions at shared ancestors implements the additive structure required by the derivative of a DAG with branching. Hence the operational algorithm equals the global adjoint pullback. $\square$

6.4 Interpretation

Backpropagation is therefore not merely a heuristic or engineering trick: in the exact differentiable setting it computes the adjoint of the program’s semantic linearisation. This is the canonical semantic baseline against which the later generalised propagation procedures are compared.

7 Neural Networks in Differential Categories

We specialise the general theory to neural networks, including feed-forward, recursive, meta-learning, and implicit architectures.

7.1 Feed-forward networks as morphisms

A feed-forward network is a composite $f = f_L \circ \dots \circ f_1$ with layers

$$f_\ell(x_\ell) = \sigma_\ell(W_\ell x_\ell + b_\ell).$$

Linear maps, affine shifts, and nonlinear activations are primitive morphisms in the chosen differentiable setting, so the network is a composite morphism.

7.2 Differential structure of layers

The layer derivative is

$$Df_\ell(x, v) = D\sigma_\ell(W_\ell x + b_\ell)(W_\ell v),$$

which is the Jacobian-vector expression used by modern AD systems.

7.3 Backpropagation as reverse action

Operationally, exact backpropagation evaluates all intermediate activations, seeds an output covector, propagates it backward through local adjoints, and accumulates parameter gradients through the same pullback. This is Theorem 6.1 specialised to the layered architecture.

7.4 Parameter differentiation

With parameters $\theta = (W_\ell, b_\ell)_\ell$ in a parameter object D_Θ , the denotation is a map

$$\text{NN} : D_\Theta \times X \rightarrow Y.$$

Differentiation with respect to the parameter argument yields the linear map whose adjoint gives the usual gradient with respect to θ .

7.5 Networks with control flow

Modern differentiable programs admit conditionals, loops, recursion, and data-dependent computation. In a domain-theoretic interpretation, recursion and loops are represented through fixed points; differentiability is preserved when the chosen fixed-point operator and semantic derivative satisfy the required compatibility assumptions.

7.6 Differentiating through optimisers

An optimiser update

$$\theta_{k+1} = \text{Opt}(\theta_k, \nabla_\theta L_k)$$

can itself be treated as a differentiable morphism. A finite training loop is then a composite computation, and meta-gradients are obtained by applying the same compositional derivative machinery to the optimiser trajectory.

7.7 Implicit layers

Implicit layers satisfy

$$z = F_\theta(z, x),$$

so z is a fixed point of $F_\theta(\cdot, x)$. Under the assumptions of the implicit function theorem,

$$D_x z = (I - \partial_z F_\theta)^{-1} \partial_x F_\theta,$$

and reverse mode applies the adjoint of this linear operator. This recovers the form used in equilibrium and implicit-layer differentiation.

8 Implicit Differentiation and Higher-Order AD

We now retain the original manuscript's higher-order and fixed-point development.

8.1 Higher-order differentials

The differential combinator can be iterated:

$$D^2 f = D(Df), \quad D^3 f = D(D^2 f), \quad \dots$$

In settings satisfying the usual higher-order axioms these derivatives are multilinear in tangent arguments, compatible with products, and obey higher-order chain rules.

8.2 Domain-theoretic higher derivatives

Given a differentiable Scott-continuous $f : D_X \rightarrow D_Y$, its second derivative can be written schematically as

$$D^2 f(x)(v_1, v_2) = D(Df(\cdot)(v_2))(x)(v_1),$$

with higher derivatives obtained by iteration when continuity and multilinearity are preserved.

8.3 Hessian- and Jacobian-vector products

Second-order information can be computed without forming a full Hessian. In finite dimensions,

$$H_f(x)v = D(\nabla f)(x)v,$$

and mixed forward/reverse constructions yield Hessian-vector and vector-Jacobian products [Pearlmutter, 1994].

8.4 Implicit differentiation

Suppose z is defined by

$$z = F_\theta(z, x).$$

Differentiating the fixed-point equation gives

$$Dz = DF_\theta(z, x)(Dz, Dx),$$

or, separating the z and x arguments,

$$(I - \partial_z F_\theta(z, x))Dz = \partial_x F_\theta(z, x).$$

Provided $I - \partial_z F_\theta$ has a continuous inverse,

$$Dz = (I - \partial_z F_\theta(z, x))^{-1} \partial_x F_\theta(z, x). \quad (8.1)$$

8.5 Reverse-mode implicit differentiation

Taking adjoints gives the reverse sensitivity operator

$$Dz^* = \partial_x F_\theta(z, x)^*(I - \partial_z F_\theta(z, x))^{-*},$$

which is the familiar implicit-gradient form.

8.6 Neural ODEs and continuous-depth models

A neural ODE solves

$$\frac{dz}{dt} = f_\theta(z, t), \quad z(0) = x.$$

The solution operator can be interpreted through an integral fixed-point equation. The corresponding adjoint sensitivity equation is a continuous-time reverse propagation law, connecting neural-ODE adjoints with the same pullback principle.

8.7 Higher-order implicit layers

Further differentiation of (8.1) produces second-order sensitivity equations with the same inverse operator $(I - \partial_z F_\theta)^{-1}$ acting on derivatives of the local Jacobians. These expressions arise in second-order meta-learning and curvature-aware implicit models.

9 Applications, Extensions, and Unified Perspective

We retain the original survey of applications and then use it as context for the later generalised propagation theory.

9.1 Differentiable programming languages

Differentiable languages and libraries such as JAX, Julia/Zygote, and PyTorch treat differentiable functions as first-class computational objects. Their semantics combine ordinary evaluation, program composition, differentiation transforms, higher-order functions, and control flow. The categorical view interprets these operations structurally rather than as unrelated compiler tricks [Abadi and Plotkin, 2020].

9.2 Attention and transformers

Scaled dot-product attention has the form

$$\text{Att}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d})V.$$

Matrix multiplication, softmax, and elementwise operations are differentiable primitives; their tangent and reverse actions compose by the same general rules.

9.3 Graph neural networks

Message-passing layers take the form

$$h_v^{(t+1)} = \phi\left(h_v^{(t)}, \{h_u^{(t)} : u \in N(v)\}\right).$$

They are naturally represented on computational graphs with shared local morphisms and repeated composition. This repeated parameter sharing later becomes important in the credit-horizon experiments.

9.4 Probabilistic models and reparameterisation

The reparameterisation

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1),$$

turns a stochastic sampling path into a deterministic differentiable map conditioned on noise, allowing ordinary AD to propagate through the deterministic transformation.

9.5 Physics-informed networks and PDE solvers

Physics-informed neural networks require derivatives of network outputs with respect to input coordinates. Higher-order AD computes these terms compositionally. More generally, scientific adjoint-state methods compute sensitivities of discretised PDE solvers by solving reverse equations that are closely related to reverse-mode AD.

9.6 Differential geometric viewpoint

On smooth manifolds, the tangent construction is the usual tangent bundle and the reverse action is cotangent pullback. Differential categories generalise the compositional algebra of these constructions beyond ordinary manifolds.

9.7 Implicit optimisation layers

For

$$z^*(x) = \arg \min_z \Phi(z, x),$$

implicit differentiation of the optimality condition gives, under regularity and nonsingularity assumptions,

$$\frac{dz^*}{dx} = -(\nabla_{zz}^2 \Phi(z^*, x))^{-1} \nabla_{zx}^2 \Phi(z^*, x),$$

and reverse mode uses the adjoint form of this solve.

9.8 Meta-learning and differentiating through training

Gradient descent defines a trajectory

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t).$$

Differentiating through the complete trajectory is again differentiation of a composite computation, giving hypergradients and meta-gradients.

9.9 Unified exact perspective

Across these applications the exact semantic principles are the same: differentiable computations compose; linearisation is functorial in the appropriate differential sense; fixed points represent recursive or implicit computations; higher-order AD iterates differentiation; and reverse mode pulls covectors backward through the compositional structure.

10 Exact-Theory Synthesis and Transition

The original manuscript concluded that algorithmic differentiation, backpropagation, differentiable programming, implicit differentiation, and scientific adjoints arise from a common mathematical core: functorial linearisation of compositional computations and adjoint propagation through that structure. Forward mode is tangent propagation; reverse mode is cotangent-style pullback when reverse structure exists; higher-order AD is iterated differentiation; and implicit differentiation comes from differentiating fixed-point equations.

That exact conclusion remains intact. The remainder of the paper asks a different question. Exact reverse AD is semantically canonical, but a learning system may be judged by a vector of costs and outcomes rather than gradient fidelity alone. We therefore enlarge the theory from exact pullback to general backward signal transport while retaining Theorem 6.1 as the zero-defect reference case.

11 Generalised backward propagation

Definition 11.1 (Propagation procedure). A *propagation procedure* P over a primal graph G consists of

$$P = (H, (S_v)_v, q, (B_{wv})_{(w,v) \in F}, (A_v)_v, (U_v)_v),$$

where:

- $H = (V, F)$ is a feedback graph, not necessarily G^{op} ;
- S_v is the backward signal space at node v ;
- $q_v \in S_v$ is a source or teaching signal;
- $B_{wv} : S_w \rightarrow S_v$ transports signals along feedback edges;
- A_v aggregates incoming signals at v ; and
- U_v maps local forward state, parameters, and s_v into a parameter update direction.

This definition intentionally does not require $S_v = X_v^*$, does not require $H = G^{\text{op}}$, and does not require B_{wv} to be a derivative. It therefore includes exact reverse AD but also random feedback, learned feedback, local losses, discrete differences, and shortcut propagation.

11.1 Linear-additive propagation

When each S_v is a vector space and aggregation is addition, stack all signals into $s \in S = \bigoplus_v S_v$. The transports define a block operator $\mathcal{B} : S \rightarrow S$. The backward field is then

$$\boxed{s = q + \mathcal{B}s.} \tag{11.1}$$

The learned update direction is

$$d_P = U(x, \theta, s). \quad (11.2)$$

In exact reverse AD, $S_v = X_v^*$, $\mathcal{B}$ is assembled from local adjoint Jacobians, and U is the corresponding local parameter pullback described in Section 6.

Theorem 11.2 (Finite Neumann flow on a DAG). *Suppose the feedback graph follows reversed edges of a finite feed-forward DAG and its longest feedback path has length D . Then, after a topological ordering, $\mathcal{B}$ is strictly triangular, hence*

$$\mathcal{B}^{D+1} = 0.$$

Consequently (11.1) has the unique solution

$$s^* = (I - \mathcal{B})^{-1}q = \sum_{j=0}^D \mathcal{B}^j q. \quad (11.3)$$

Proof. A product of k nonzero feedback blocks corresponds to a feedback path of length k . No path is longer than D , so every block of $\mathcal{B}^{D+1}$ vanishes. Therefore

$$(I - \mathcal{B}) \sum_{j=0}^D \mathcal{B}^j = I - \mathcal{B}^{D+1} = I,$$

which proves both existence and uniqueness. $\square$

Corollary 11.3 (Backpropagation as triangular solve). *Classical reverse-topological backpropagation computes (11.3) by exact substitution in the triangular system $(I - \mathcal{B})s = q$.*

Corollary 11.4 (Jacobi propagation). *Starting from $s^{(0)} = 0$, the parallel iteration*

$$s^{(k+1)} = q + \mathcal{B}s^{(k)} \quad (11.4)$$

obeys

$$s^{(K)} = \sum_{j=0}^{K-1} \mathcal{B}^j q,$$

and becomes exact after at most $D + 1$ iterations.

Thus the familiar numerical-analysis distinction between direct and iterative linear solvers [Saad, 2003] has a direct credit-assignment analogue. Reverse traversal is an exact triangular solve; synchronous message passing is Jacobi iteration; asynchronous or ordered variants correspond to Gauss–Seidel-like schedules; relaxation and preconditioning suggest further propagation algorithms.

11.2 Finite credit horizons

Definition 11.5 (Credit horizon). For $K \leq D$, the exact-transport K -horizon field is

$$s_K = \sum_{j=0}^K \mathcal{B}^j q. \quad (11.5)$$

It assigns the current example’s output credit only through feedback paths of length at most K .

Proposition 11.6 (Truncation error). *For any submultiplicative norm,*

$$\|s^* - s_K\| \leq \sum_{j=K+1}^D \|\mathcal{B}\|^j \|q\|. \quad (11.6)$$

If an infinite or cyclic system satisfies $\|\mathcal{B}\| < 1$, then

$$\|s^* - s_K\| \leq \frac{\|\mathcal{B}\|^{K+1}}{1 - \|\mathcal{B}\|} \|q\|. \quad (11.7)$$

The bound concerns instantaneous signal fidelity. It says nothing yet about the eventual learning trajectory, because parameters are updated repeatedly and shared parameters can receive credit at different graph depths on different examples.

11.3 Shortcut feedback

A feedback graph need not be the reverse primal graph. Direct output-to-hidden edges can reduce the feedback critical path to one, as in direct feedback alignment [Nøkland, 2016]. In the flow equation this changes $\mathcal{B}$, not merely the order used to solve it. Hybrid schemes may combine exact local transport with shortcut messages.

11.4 Cyclic and implicit computations

For recurrent, equilibrium, or implicit systems, $\mathcal{B}$ need not be nilpotent. If $I - \mathcal{B}$ is invertible,

$$s^* = (I - \mathcal{B})^{-1} q.$$

When $\rho(\mathcal{B}) < 1$, iterative propagation converges under standard conditions. This places implicit differentiation, adjoint-state methods, and equilibrium-style learning within the same signal-equation view. The numerical method used to solve the backward fixed point becomes part of the propagation procedure rather than an implementation detail.

12 Alternative propagation calculi

The transport operator can be changed in several logically independent ways.

12.1 Alternative local derivatives

Replace an exact local derivative Df by a rule $\tilde{D}f$. Examples include finite differences, secants, subgradient selections, straight-through derivatives, quantised derivatives, and Boolean or discrete differences. A forward operation and its backward rule need not therefore belong to the same differential calculus.

For a composable pair $f : X \rightarrow Y$, $g : Y \rightarrow Z$, define a reverse propagation assignment R and its *composition defect*

$$\Delta_R(f, g; x) = \|R_{g \circ f}(x) - R_f(x) \circ R_g(f(x))\|. \quad (12.1)$$

Exact reverse differentiation has zero defect. Surrogate rules can have nonzero defect yet still produce useful update directions.

12.2 Random and learned transports

Feedback alignment replaces transposed forward weights by fixed random feedback weights [Lillicrap et al., 2016]; direct feedback alignment sends output errors directly to hidden layers [Nøkland, 2016]. Synthetic gradients learn an estimator of a future backward signal, allowing modules to update without waiting for full reverse execution [Jaderberg et al., 2017]. These procedures alter the transport map, the graph, or even the source signal.

12.3 Local targets and semantic supervision

A more radical case removes inter-level credit transport entirely. Each node receives a local target and updates from a local loss. In a compositional symbolic task, subexpression semantics provide an unusually clean version of this idea: every AST node can be labelled by the truth value of the subformula rooted there. Such a rule has zero feedback path length while still training shared modules that are reused globally.

12.4 Perturbation of the signal solver

Proposition 12.1 (Transport perturbation bound). *Suppose both $I - \mathcal{B}$ and $I - \tilde{\mathcal{B}}$ are invertible. For the same source q , let*

$$s = (I - \mathcal{B})^{-1}q, \quad \tilde{s} = (I - \tilde{\mathcal{B}})^{-1}q.$$

Then

$$\|s - \tilde{s}\| \leq \|(I - \mathcal{B})^{-1}\| \|\mathcal{B} - \tilde{\mathcal{B}}\| \|(I - \tilde{\mathcal{B}})^{-1}\| \|q\|. \quad (12.2)$$

Proof. Use the resolvent identity

$$(I - \mathcal{B})^{-1} - (I - \tilde{\mathcal{B}})^{-1} = (I - \mathcal{B})^{-1}(\mathcal{B} - \tilde{\mathcal{B}})(I - \tilde{\mathcal{B}})^{-1}$$

and apply the operator norm. $\square$

This separates three sources of backward error: transport mismatch, conditioning of the exact flow, and conditioning of the surrogate flow.

13 Learning with inexact propagated directions

The exact-gradient property is stronger than the property needed for a single descent step.

Theorem 13.1 (Positive-alignment descent). *Let $L : \mathbb{R}^p \rightarrow \mathbb{R}$ have β -Lipschitz gradient, let $g = \nabla L(\theta)$, and let a propagation procedure produce a nonzero direction d . For*

$$\theta^+ = \theta - \eta d,$$

we have

$$L(\theta^+) \leq L(\theta) - \eta \langle g, d \rangle + \frac{\beta \eta^2}{2} \|d\|^2. \quad (13.1)$$

If $\langle g, d \rangle > 0$, then every

$$0 < \eta < \frac{2 \langle g, d \rangle}{\beta \|d\|^2} \quad (13.2)$$

produces strict descent.

Proof. Equation (13.1) is the standard descent lemma [Nocedal and Wright, 2006]. The right-hand side is strictly below $L(\theta)$ exactly when the negative linear term dominates the quadratic remainder, which yields (13.2). $\square$

Corollary 13.2 (Relative-error guarantee). *If*

$$\|d - g\| \leq \rho \|g\|, \quad 0 \leq \rho < 1, \quad (13.3)$$

then

$$\langle g, d \rangle \geq (1 - \rho) \|g\|^2, \quad \|d\| \leq (1 + \rho) \|g\|,$$

and every

$$0 < \eta < \frac{2(1-\rho)}{\beta(1+\rho)^2} \quad (13.4)$$

(up to the usual scaling by $\|g\|$ absorbed through the displayed inequalities) is sufficient under the conservative bound obtained from Theorem 13.1.

More precisely, substituting the two inequalities into (13.2) gives the stated dimensionless coefficient multiplying the reciprocal smoothness scale. The key point is qualitative: exact equality $d = g$ is unnecessary. The straight-through literature similarly studies surrogate directions that correlate with unavailable true gradients [Yin et al., 2019].

Proposition 13.3 (First-order optimality of the exact gradient). *Among directions satisfying $\|d\| = c$, the direction parallel to g maximises the first-order decrease $\langle g, d \rangle$.*

Proof. By Cauchy–Schwarz, $\langle g, d \rangle \leq \|g\| c$, with equality iff d is a positive scalar multiple of g . $\square$

This proposition explains the privileged status of the exact gradient under a very specific objective: first-order loss decrease at fixed update norm and zero accounting for the cost of producing the direction. It does not establish global optimality once propagation itself has a cost.

14 Propagation optimality

Definition 14.1 (Propagation performance vector). For a procedure P , define a vector

$$\mathcal{O}(P) = (R_{\text{terminal}}, G_{\text{systematic}}, T_{\varepsilon}, C_{\text{throughput}}, M, D_{\text{fb}}, N_{\text{transport}}, E_{\text{grad}}, L_{\text{locality}}, R_{\text{robust}}), \quad (14.1)$$

with coordinates interpreted below. Signs are chosen so that a Pareto ordering can be imposed consistently.

The coordinates correspond to inequivalent notions of optimality:

Semantic or pullback optimality. Minimise $E_{\text{grad}} = \|d_P - g\|$, or an operator-level cotangent error. Exact reverse AD has zero error.

Descent optimality. Maximise $\langle g, d_P \rangle$ or cosine alignment subject to an update-norm constraint.

Budget optimality. For a compute, memory, or communication budget B , minimise expected terminal risk.

Time-to-criterion optimality. Minimise wall-clock time or updates to reach a stated held-out accuracy or logical criterion.

Latency optimality. Minimise feedback critical path or synchronisation rounds. Direct feedback can be latency-optimal while semantically inexact.

Memory optimality. Minimise retained forward state, checkpointing, or recomputation cost.

Locality optimality. Minimise dependence on nonlocal activations, weight transport, or global synchronisation [Akrou et al., 2019].

Numerical optimality. Minimise fixed-point convergence factor or signal-solver residual.

Robustness optimality. Minimise degradation under noisy, stale, quantised, or missing messages.

Logical optimality. Maximise truth accuracy, theoremhood, entailment, rewrite consistency, and systematic extrapolation in symbolic tasks.

No scalar ordering is canonical across these coordinates. The natural general notion is Pareto optimality or, when an application supplies explicit prices, minimisation of a scalarised objective.

14.1 Budget-dependent credit depth

For a training maximum depth d_{tr} , a target test depth d_{te} , and budget B , define

$$K^*(d_{\text{tr}}, d_{\text{te}}, B) = \arg \max_K \mathbb{E}_{\text{seed}} \left[A_{d_{\text{te}}} \left(\theta_{\tau_K(B)}^{(K)} \right) \right], \quad (14.2)$$

where $\tau_K(B)$ is the latest training checkpoint affordable by horizon K . Ties are broken toward smaller K . This definition asks for the horizon that learns best under the budget, not the horizon that best approximates the instantaneous gradient.

15 Propositional logic as a credit-assignment laboratory

Propositional logic provides several advantages for studying propagation. Ground-truth semantics are exact; every valuation can be enumerated for a small variable set; formula depth is directly controlled; logically equivalent expressions provide invariance tests; theoremhood and entailment are objective; and shared logical operators create repeated compositional modules at many depths.

We use Boolean values $\{0, 1\}$ and the operators

$$\neg, \quad \wedge, \quad \vee, \quad \oplus, \quad \rightarrow, \quad \leftrightarrow.$$

Implication is evaluated as $\neg p \vee q$, and equivalence as the negation of XOR.

16 Experiment I: fixed truth-vector learning

16.1 Setup

The first experiment asks only whether strongly different backward rules can learn the same finite semantic task. Inputs are all $2^6 = 64$ valuations of six propositions. The target is a 32-dimensional vector of formula truth values containing negation, conjunction, disjunction, XOR, implication, equivalence, De Morgan identities, contraposition, modus ponens, hypothetical syllogism, distributivity, chained implications, and mixed formulas. A fully connected network with dimensions

$$6 \rightarrow 64 \rightarrow 64 \rightarrow 64 \rightarrow 32$$

uses tanh hidden activations and sigmoid outputs. Each seed uses 48 valuations for training and 16 for held-out testing. Results below average five seeds after 2500 epochs.

The compared procedures are exact backpropagation, full Jacobi propagation, two-hop Jacobi truncation, a finite-difference tanh derivative, a straight-through derivative, feedback alignment, direct feedback, hybrid exact/direct shortcuts, and trainable local heads.

16.2 Results

The main observation is not a ranking. It is the separation between exact gradient fidelity and the ability to fit the semantic task. Feedback alignment and direct feedback both achieve perfect training bit accuracy despite mean gradient cosines of approximately 0.14 and 0.29, respectively. Exact gradient identity is therefore not necessary for successful learning on this benchmark.

17 Experiment II: compositional formula learning

17.1 AST model

The stronger experiment receives a propositional formula abstract-syntax tree (AST) and a valuation, and predicts the formula’s truth value. There are four variables, so every formula is

Table 1: Fixed truth-vector experiment, five-seed means. “Cosine” is mean cosine similarity of the propagated update with the exact gradient.

Method	Train bit acc.	Test bit acc.	Cosine
Exact BP	100.0%	93.75%	1.000
Full Jacobi	100.0%	93.75%	1.000
Two-hop truncation	100.0%	93.52%	0.903
Finite difference	100.0%	94.14%	1.000
Straight-through	99.64%	87.89%	0.150
Feedback alignment	100.0%	92.66%	0.144
Direct feedback	100.0%	92.89%	0.287
Hybrid shortcut	100.0%	94.38%	0.941
Local heads	100.0%	95.12%	0.583

evaluated on all 16 valuations. The model has hidden dimension $H = 12$ and shared modules:

$$\begin{aligned}
h_{\text{VAR}} &= \tanh(xW_{\text{VAR}} + b_{\text{VAR}}), \\
h_{\neg} &= \tanh(h_a W_{\neg} + b_{\neg}), \\
h_{\circ} &= \tanh([h_a, h_b]W_{\circ} + b_{\circ}), \quad \circ \in \{\wedge, \vee, \oplus, \rightarrow, \leftrightarrow\}, \\
\hat{y} &= \sigma(h_{\text{root}}W_{\text{out}} + b_{\text{out}}).
\end{aligned}$$

Thus the same learned AND, OR, implication, and other modules are reused at every syntactic depth.

Training uses 160 randomly generated formulas of depths 1–3, mini-batches of 10 formulas, Adam, learning rate 0.008, and 80 epochs. Held-out tests include 80 unseen formulas of depths 1–3 and 50 formulas each at depths 4, 5, and 6. Additional suites test theoremhood, entailment, and agreement under standard equivalence rewrites.

17.2 Backward procedures

The AST experiment compares:

- exact adjoint propagation;
- a numerical finite-difference derivative for $\tanh$;
- a straight-through derivative $d \tanh / dz := 1$;
- fixed random feedback alignment;
- direct output-to-node feedback;
- a hybrid of exact local transport and direct shortcuts;
- exact transport truncated to two feedback edges; and
- *local semantic propagation*: each AST node receives the exact truth label of its own subformula and trains from a node-local loss, with no inter-level feedback transport.

17.3 Accuracy, logical generalisation, and speed

Core optimisation time excludes held-out evaluation and gradient-cosine diagnostics. Throughput counts formula passes per second, each pass containing all 16 valuations. Time-to-threshold is measured from held-out same-depth evaluation sampled every five epochs.

The two-hop truncated flow is approximately 9.4% faster in core training time than exact BP and reaches 95% same-depth held-out accuracy about 16.7% sooner, while preserving similar depth-6 accuracy. Local semantic propagation is approximately 46.6% slower than exact BP but gives the strongest systematic performance.

Gradient fidelity and logical generalisation are visibly distinct (Figure 1). Local semantic propagation has mean cosine only about 0.589 with the exact gradient while reaching approximately 95.8% at depth 6 and 99.7% on theoremhood and entailment. Direct feedback has lower

Table 2: Compositional AST experiment, three-seed means. Times are implementation-specific Python/NumPy wall-clock measurements and should be interpreted comparatively.

Method	Train s	Formulas/s	Time to 95%	Unseen	Depth 6	Theorem	Entail.
Exact BP	2.319	5521	0.468	98.57%	90.83%	95.49%	96.18%
Finite difference	2.378	5384	0.449	98.62%	91.00%	95.49%	96.53%
Straight-through	2.194	5837	–	72.16%	61.67%	67.01%	55.90%
Feedback alignment	2.184	5861	–	88.07%	73.04%	74.65%	75.69%
Direct feedback	2.274	5630	–	80.57%	64.88%	67.71%	73.26%
Hybrid shortcut	2.472	5179	0.461	98.49%	90.38%	93.75%	95.14%
Two-hop truncation	2.101	6094	0.390	98.85%	90.13%	94.10%	96.88%
Local semantic	3.400	3765	0.562	99.82%	95.75%	99.65%	99.65%

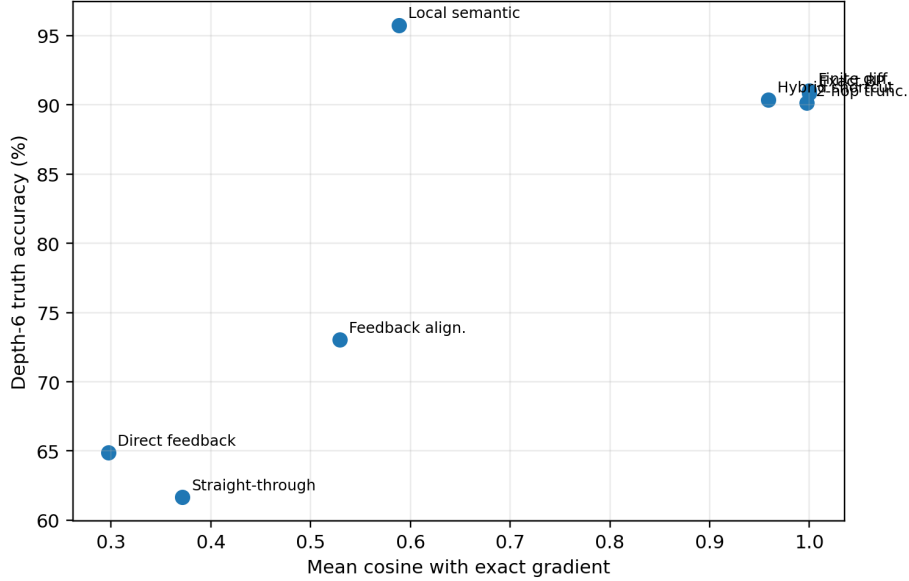


Figure 1: Gradient alignment versus systematic depth-6 truth accuracy in the AST experiment. High gradient fidelity is neither necessary nor sufficient to explain the full ranking of systematic generalisation.

gradient alignment and substantially weaker systematic performance, showing that inexactness alone is not beneficial: the structure of the teaching signal matters.

18 Experiment III: optimal credit horizons

18.1 Design

We now hold the local derivative exact and vary only the instantaneous credit horizon K . Training maximum depths are $d \in \{2, 3, 4\}$. For each d , the experiment uses 160 training formulas, 50 same-depth held-out formulas, 30 formulas at each of depths $d + 1, d + 2, d + 3$, all 16 valuations, 80 epochs, and three parameter seeds. Checkpoints occur at epochs 10, 20, 40, and 80 in the reported sweep.

The structural work proxy is

$$W_K = \text{extformulapasses} \times (1 + \text{reached nodes} + \text{transported edges}). \quad (18.1)$$

The additive 1 accounts for the output head. This proxy is deliberately simple; it complements rather than replaces wall-clock timing.

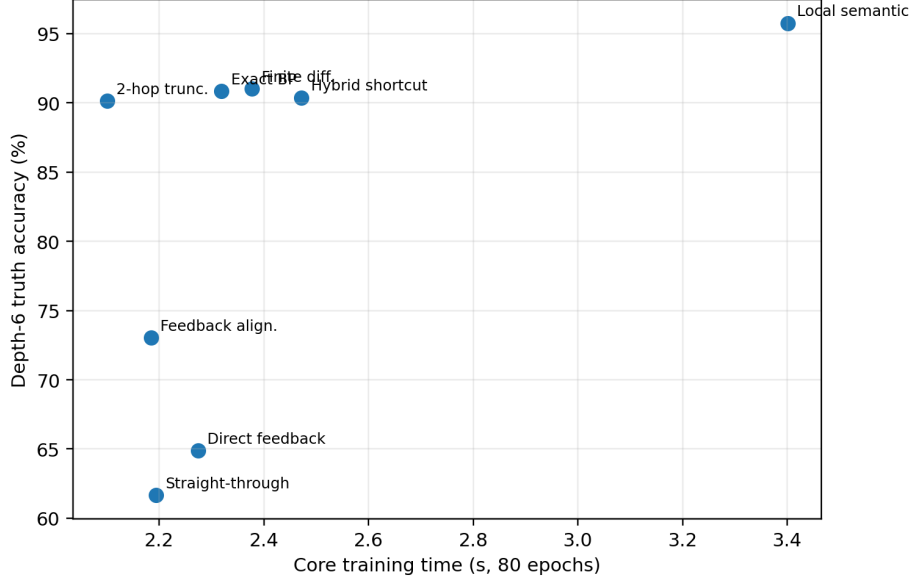


Figure 2: Core optimisation time versus depth-6 accuracy. The plotted frontier makes the trade-off between speed and systematicity explicit.

18.2 Final results

For the hardest target $d_{te} = d_{tr} + 3$, the full-budget empirical optima are

$$\boxed{K^*(2, 5) = 1, \quad K^*(3, 6) = 1, \quad K^*(4, 7) = 3.} \quad (18.2)$$

The same choices are selected at every sampled 25%, 50%, 75%, and 100% wall-clock and structural-work budget fraction in the experiment.

At training depth 2, $K = 1$ uses 38.6% less structural work and about 9.7% less training time than full $K = 2$, while improving depth-5 accuracy by 4.65 percentage points. At training depth 3, $K = 1$ uses 54.1% less structural work and about 17.3% less training time than full $K = 3$, while improving depth-6 accuracy by 2.36 points. At depth 4, $K = 3$ uses 15.5% less structural work and about 4.5% less time than $K = 4$, with essentially the same mean depth-7 accuracy.

18.3 Variance and statistical caution

The depth+3 standard deviation is approximately 3.18 percentage points for depth-2/ $K = 1$, 2.76 points for depth-3/ $K = 1$, and 15.07 points for depth-4/ $K = 3$. The last regime is therefore materially less stable. Seed-wise hardest-target winners are $K = (2, 1, 1)$ for training depth 2, $K = (1, 1, 2)$ for depth 3, and $K = (3, 1, 3)$ for depth 4. Larger seed and dataset sweeps are required before making a precise scaling claim for K^* .

19 Experiment IV: MNIST with a published LeNet-5 configuration

The propositional experiments deliberately use a structured domain with reusable logical operators. To test whether the main conclusions survive outside that setting, we next use the standard MNIST handwritten-digit benchmark and a published LeNet-5-style configuration. The original LeNet family and its use for handwritten document recognition are described by [LeCun et al. \[1998\]](#). For a modern reproducible configuration we follow the LeNet-5 setup reported by

Table 3: Credit-horizon sweep at 80 epochs. Accuracy is on formulas three levels deeper than the maximum training depth.

Train depth	K	Same-depth	Depth +3	Grad. cosine	Work
2	0	93.75%	66.88%	0.185	25,600
2	1	99.50%	92.99%	0.976	74,080
2	2 exact	99.17%	88.33%	1.000	120,640
3	0	80.50%	62.36%	0.181	25,600
3	1	97.75%	90.42%	0.943	73,760
3	2	97.87%	85.56%	0.996	125,280
3	3 exact	98.33%	88.06%	1.000	160,800
4	0	71.37%	61.60%	0.400	25,600
4	1	97.42%	84.65%	0.992	74,720
4	2	95.87%	85.28%	0.999	132,320
4	3	96.42%	86.94%	1.000	184,480
4	4 exact	96.42%	86.74%	1.000	218,240

Table 4: MNIST/LeNet-5-style benchmark, three seeds and three epochs. T_{95} and T_{98} are mean optimisation times to reach 95% and 98% test accuracy among seeds that reach the threshold. Parentheses show the number of successful seeds where not all three reach the threshold.

Method	Test acc.	Time (s)	Ex./s	T_{95}	T_{98}	Global cos.	Active blocks
Exact BP	98.48 ± 0.07	25.94	6944	8.50	20.32	1.000	1.00
$K = 4$	98.07 ± 0.11	19.42	9322	5.86	12.06 (2/3)	0.999	0.80
$K = 3$	96.78 ± 0.12	13.15	13684	8.73	–	0.992	0.60
$K = 2$	80.09 ± 3.80	9.56	18918	–	–	0.612	0.40
$K = 1$	32.53 ± 6.64	9.53	18954	–	–	0.563	0.20
DFA	95.39 ± 0.83	23.71	7753	21.30 (2/3)	–	−0.001	1.00
Local	98.39 ± 0.23	25.95	7047	7.49	18.67	0.146	1.00

Kim and Kim [2022]: two 5×5 convolutional stages with 6 and 16 channels, followed by fully connected layers of widths 120, 84, and 10, ReLU activations, a softmax/cross-entropy objective, mini-batches of 128, and Adam with learning rate 10^{-3} [Kingma and Ba, 2015]. The published study reports a reproduced MNIST test accuracy of $99.10\% \pm 0.05\%$ at convergence; our purpose is different, so we use only three epochs and measure the early speed–accuracy trade-off.

19.1 Propagation procedures

There are five trainable blocks, ordered from input to output as

$$C_1 \rightarrow C_2 \rightarrow F_1 \rightarrow F_2 \rightarrow F_3.$$

Exact backpropagation updates all five. A suffix horizon K updates only the last K blocks on that mini-batch. We compare $K = 1, 2, 3, 4$, exact backpropagation ($K = 5$), direct feedback alignment (DFA), and local auxiliary-error propagation. The latter two provide signals to all blocks without using the exact reverse chain.

19.2 Early learning and training speed

Table 4 reports three-seed means after three epochs. Timing measures core optimisation time on the experimental CPU process and should be interpreted comparatively, not as a hardware-independent benchmark.

The external benchmark both confirms and qualifies the propositional results. First, exact gradient fidelity is again not necessary for useful learning. Local propagation nearly matches

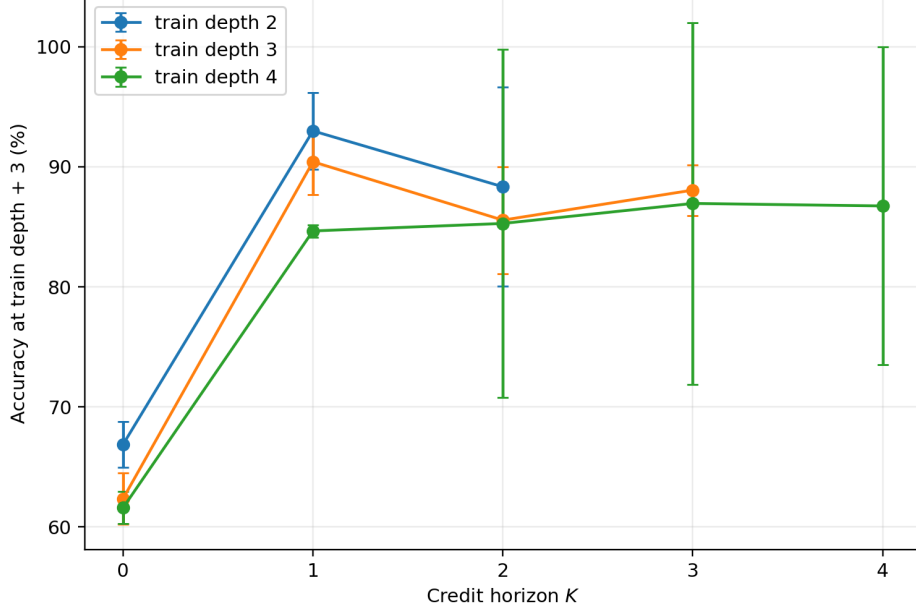


Figure 3: Hard extrapolation accuracy versus credit horizon. Error bars are standard deviations over three seeds. The depth-4 ranking has substantial seed variance.

exact BP after three epochs despite an initial global gradient cosine of only 0.146. DFA begins with an essentially orthogonal global direction, with mean cosine -0.001 , yet reaches 95.39% mean test accuracy. Second, fixed truncation has a much sharper failure mode than in the logical AST: $K = 1$ and $K = 2$ never send any signal to the early convolutional blocks and perform poorly. Third, a moderate truncation can still be attractive: $K = 4$ loses only 0.41 percentage points relative to exact BP while reducing mean optimisation time by roughly 25% in this short run.

19.3 Global cosine is not enough

The MNIST experiment also exposes a weakness in the usual single-vector alignment diagnostic. Write the exact and propagated parameter directions blockwise as $g = (g_1, \dots, g_L)$ and $d = (d_1, \dots, d_L)$. In addition to global cosine, define

$$A_{\text{bal}}(g, d) = \frac{1}{L} \sum_{\ell=1}^L a_{\ell}, \quad a_{\ell} = \begin{cases} \frac{\langle g_{\ell}, d_{\ell} \rangle}{\|g_{\ell}\| \|d_{\ell}\|}, & \|d_{\ell}\| > 0, \\ 0, & \|d_{\ell}\| = 0, \end{cases} \quad (19.1)$$

and instantaneous structural coverage

$$C_{\text{inst}}(d) = \frac{1}{L} \sum_{\ell=1}^L \mathbf{1}\{\|d_{\ell}\| > 0\}. \quad (19.2)$$

Table 5 and Figure 7 show that $K = 3$ has global cosine 0.992 but layer-balanced cosine only 0.600 because both convolutional blocks are inactive. DFA and local propagation are poorly aligned globally but have complete instantaneous structural coverage. Gradient-vector fidelity and credit accessibility are therefore distinct quantities.

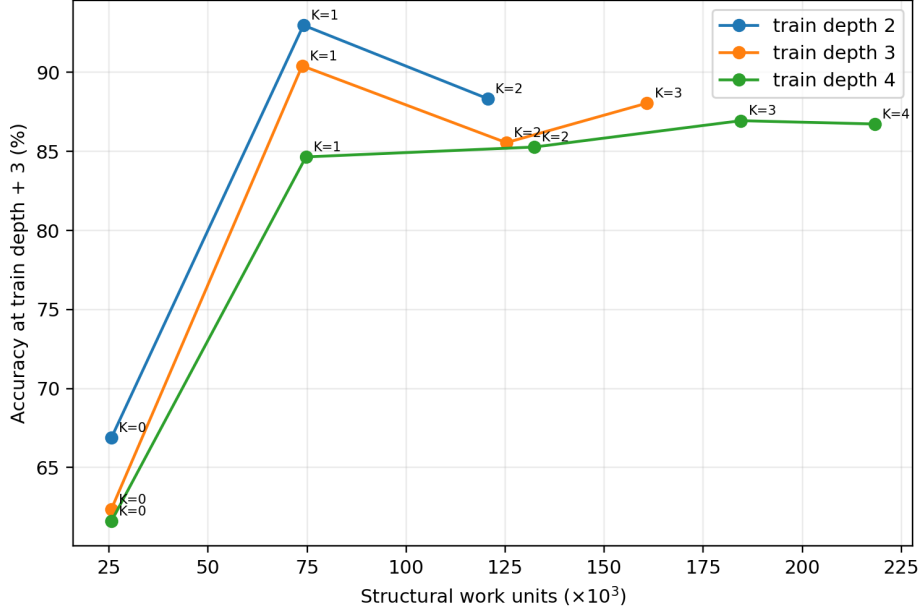


Figure 4: Systematic depth extrapolation versus the structural work proxy. Exact full-depth propagation is not the unique point on the performance frontier.

Table 5: Initial MNIST propagation diagnostics, averaged over three seeds.

Method	Global cosine	Layer-balanced cosine	Active-layer fraction
Exact BP	1.000	1.000	1.00
$K = 4$	0.999	0.800	0.80
$K = 3$	0.992	0.600	0.60
$K = 2$	0.612	0.400	0.40
$K = 1$	0.563	0.200	0.20
DFA	-0.001	0.209	1.00
Local	0.146	0.237	1.00

19.4 Temporal credit coverage

For a training trajectory, define the temporal credit coverage of block ℓ by

$$C_\ell(T; P) = \frac{1}{T} \sum_{t=1}^T \mathbf{1}\{\|d_{\ell,t}^P\| > 0\}, \quad (19.3)$$

and the maximum starvation gap

$$G_\ell(T; P) = \max\{\text{number of consecutive updates with } \|d_{\ell,t}^P\| = 0\}. \quad (19.4)$$

The propositional AST and LeNet results can now be reconciled. In the AST, shared logical-operator parameters recur at multiple syntactic depths, so a parameter can receive credit on one formula even when another occurrence lies outside the current horizon. In sequential LeNet, an early block that is outside a fixed suffix horizon is outside it on every example. Thus useful short-horizon learning depends not merely on graph depth but on *credit coverage over the data and training trajectory*.

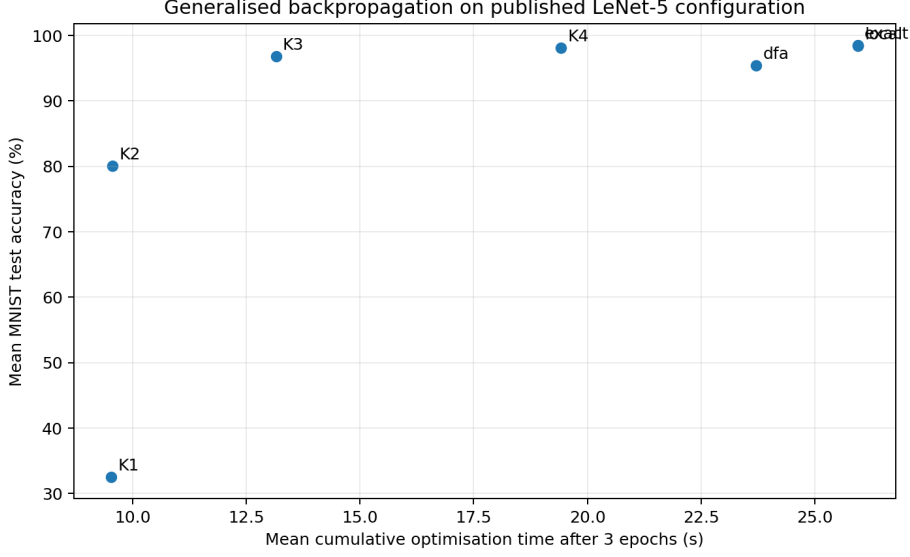


Figure 5: MNIST speed–accuracy frontier after three epochs. Exact BP achieves the highest mean accuracy, while $K = 4$ and $K = 3$ reduce optimisation time substantially.

20 Experiment V: coverage-constrained adaptive sparse propagation

The MNIST failure of small fixed horizons motivates a sparse rule that keeps the cheap $K = 3$ suffix most of the time but guarantees that no early layer remains untrained indefinitely.

Definition 20.1 (Coverage-constrained suffix policy). Consider L trainable blocks ordered from input to output. A suffix horizon K_t updates exactly the last K_t blocks on update t . Let $s_\ell(t)$ be the number of consecutive preceding updates on which block ℓ received no credit, and let $\bar{G}_\ell$ be a maximum admissible starvation gap. The policy is feasible if $s_\ell(t) < \bar{G}_\ell$ for every ℓ and t .

For the five-block LeNet experiment, the controller uses base horizon $K = 3$, requires C_2 to receive credit at least once every four mini-batches and C_1 at least once every eight mini-batches, and may propagate further after an unusually high-loss mini-batch. The deadline rule always dominates the optional loss-surprise escalation.

Proposition 20.2 (Pointwise minimal deadline horizon). *Let block ℓ require suffix horizon $r_\ell = L - \ell + 1$ to receive credit. At update t , let D_t be the set of blocks whose starvation deadline would be violated if they were skipped. Then*

$$K_t^{\min} = \max \left\{ K_{\text{base}}, \max_{\ell \in D_t} r_\ell \right\} \quad (20.1)$$

is the smallest suffix horizon that satisfies all current deadlines. Hence, among suffix policies subject to the same current deadline constraints, the deadline component of the controller is pointwise work-minimal.

The optional surprise rule can deliberately choose $K_t > K_t^{\min}$; it trades additional current work for a possible learning benefit. Equation (20.1) supplies the hard coverage guarantee.

20.1 Adaptive results

The controller uses a mean horizon of only 3.562 out of 5, corresponding to 71.24% of the exact suffix-block work proxy. Nevertheless it reaches $98.11\% \pm 0.17\%$ mean test accuracy after three

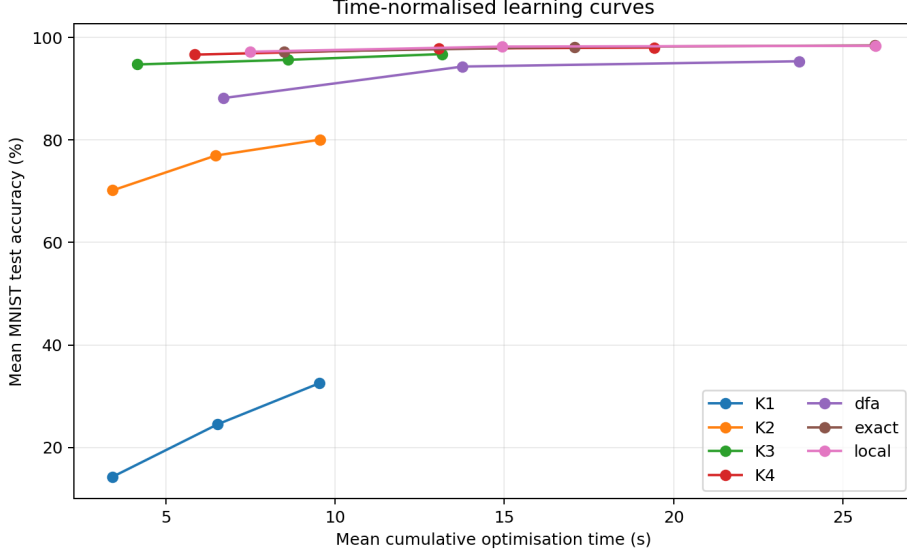


Figure 6: Time-normalised MNIST learning curves. The comparison is intentionally short-horizon and measures early learning rather than converged benchmark performance.

Table 6: Coverage-constrained adaptive propagation versus the strongest fixed-horizon baselines. Structural fraction is mean traversed suffix blocks divided by five.

Method	Test acc.	Time (s)	Mean K	Structural frac.	C_1 freq.	C_2 freq.
Exact BP	98.48 ± 0.07	25.94	5.000	1.000	1.000	1.000
$K = 4$	98.07 ± 0.11	19.42	4.000	0.800	0.000	1.000
$K = 3$	96.78 ± 0.12	13.15	3.000	0.600	0.000	0.000
Adaptive	98.11 ± 0.17	20.17	3.562	0.712	0.205	0.357

epochs, essentially tying fixed $K = 4$ and approaching exact BP. The first convolution receives direct credit on only 20.52% of mini-batches and the second on 35.70%.

20.2 Backward-kernel microbenchmark

To separate reverse-computation cost from data loading, forward execution, optimizer updates, and evaluation, we additionally time forward and backward kernels over 200 mini-batches. Table 7 reports the resulting single-process CPU microbenchmark. The adaptive controller reduces measured backward-kernel time by approximately 44.8% relative to fixed $K = 4$ and 72.2% relative to exact BP. End-to-end epoch time is noisier and does not show the same proportional gain, because the current Python implementation adds controller and detached-forward overhead.

Interpretation. The experiment supports a revised principle: *global instantaneous backpropagation is unnecessary provided every learnable block receives sufficiently frequent, sufficiently informative credit over time*. It is therefore useful to view backward computation itself as a schedulable resource. A natural constrained objective is

$$\min_P \mathbb{E} \left[\sum_{t=1}^T c(K_t) \right] \quad \text{subject to} \quad G_\ell(T; P) \leq \bar{G}_\ell \quad \forall \ell, \quad \mathcal{R}_T(P) \leq \varepsilon, \quad (20.2)$$

where $c(K)$ prices reverse work and $\mathcal{R}_T$ is a chosen learning or generalisation criterion.

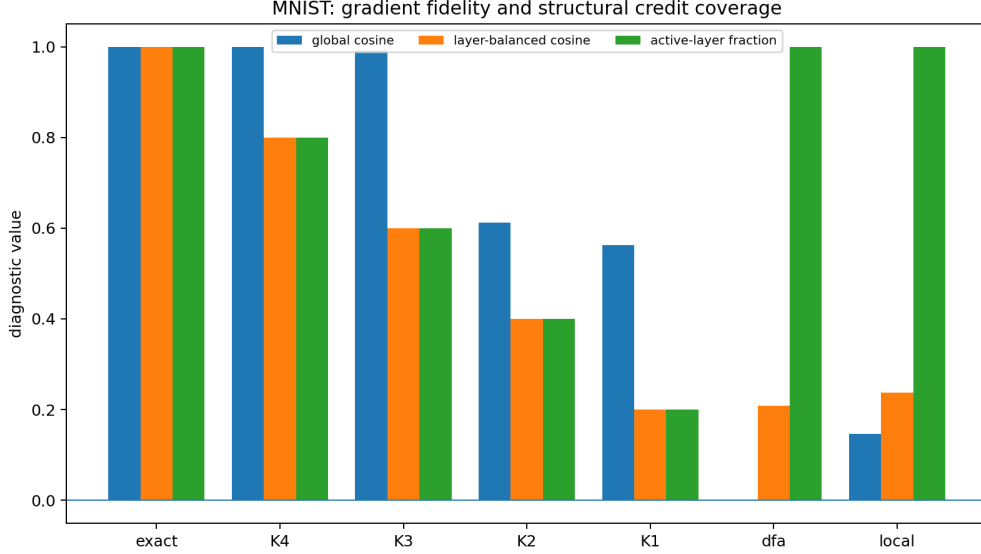


Figure 7: Global gradient cosine, layer-balanced cosine, and active-layer fraction on MNIST at initialisation. Near-perfect global cosine can conceal complete starvation of early blocks.

Table 7: Backward-kernel microbenchmark over 200 MNIST mini-batches.

Method	Mean K	Forward (ms)	Backward (ms)	Forward+backward (ms)
Exact BP	5.000	3.651	11.110	14.761
$K = 4$	4.000	4.863	5.591	10.454
$K = 3$	3.000	3.665	1.503	5.169
Adaptive	3.375	3.497	3.089	6.586

21 Two coupled flows in learning

The horizon experiment suggests a conceptual distinction that is invisible if learning is identified with a single reverse pass.

For one example, the within-example credit field is

$$q + \mathcal{B}q + \mathcal{B}^2q + \cdots + \mathcal{B}^Kq. \quad (21.1)$$

Across examples, the parameters evolve by

$$\theta_{t+1} = \theta_t - \eta_t \tilde{g}_K(\theta_t; z_t). \quad (21.2)$$

The second flow changes the forward and backward operators encountered on later examples. When modules are shared across graph levels, a parameter occurrence that is outside the credit horizon in one formula may appear close to the output in another formula. Global competence can therefore be accumulated longitudinally through shared parameters even if each individual example supplies only local instantaneous credit.

Conjecture 21.1 (Global learning without global instantaneous credit). *In compositional models with sufficiently recurrent parameter sharing across structural depths, there exist data distributions and finite horizons $K < D$ for which the longitudinal parameter dynamics learn globally compositional rules as well as or better than full instantaneous credit propagation under equal finite compute.*

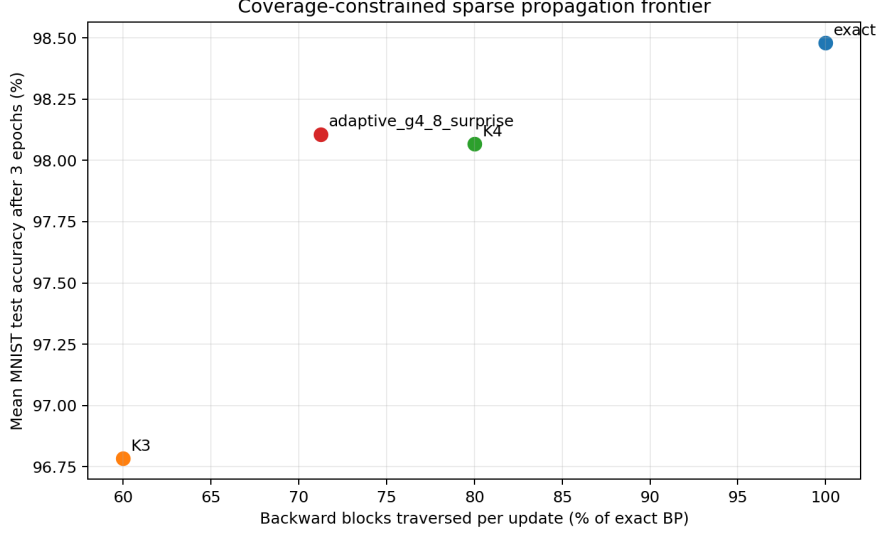


Figure 8: Coverage-constrained sparse propagation on the structural-work/accuracy plane. The adaptive rule recovers most of the $K = 4$ accuracy while traversing fewer backward blocks on average.

The propositional experiments provide evidence for the conjecture but not a proof. They also suggest that K can act as a regularisation parameter. A schematic decomposition is

$$\mathcal{R}(K, B) = \mathcal{B}_{\text{credit}}(K) + \mathcal{V}_{\text{credit}}(K) + \lambda \mathcal{C}(K, B), \quad (21.3)$$

where small K creates credit bias or starvation, large K may increase variance or over-couple updates to the training-depth distribution, and $\mathcal{C}$ is computational cost. An interior optimum is therefore plausible even when exact gradients are available.

22 Adaptive propagation depth and learned credit schedules

The fixed-horizon and coverage-constrained experiments motivate two complementary adaptive principles.

First, if several nested horizons can be evaluated or estimated, define a marginal shell statistic

$$\widehat{\Delta}_{K,t} = \frac{\|d_{K+1,t} - d_{K,t}\|}{\|d_{K,t}\| + \epsilon}. \quad (22.1)$$

A residual-style procedure can choose

$$K_t = \min \left\{ K : \widehat{\Delta}_{K,t} \leq \varepsilon_t \right\}, \quad (22.2)$$

possibly subject to a hard time or communication budget.

Second, the MNIST experiment shows that a purely marginal rule must be constrained by credit coverage. Combining the two ideas yields

$$K_t = \max \{ K_t^{\text{deadline}}, K_t^{\text{marginal}}, K_t^{\text{surprise}} \}, \quad (22.3)$$

where the first term guarantees bounded starvation and the other terms spend additional reverse work only when its estimated marginal value is high.

A more economic rule estimates loss reduction per unit extra backward cost,

$$V_{K,t} = \frac{\widehat{\Delta}_{L_{K \rightarrow K+1,t}}}{\widehat{C}_{K+1,t} - \widehat{C}_{K,t}}, \quad (22.4)$$

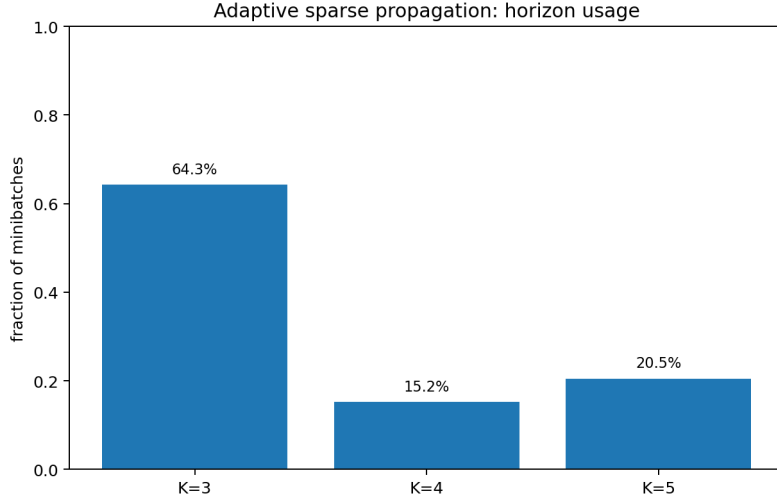


Figure 9: Horizon usage of the adaptive controller after three epochs, averaged over the three final runs. Full $K = 5$ propagation is used on only about one fifth of updates.

and continues propagation while $V_{K,t}$ exceeds an application-dependent price. The coverage gaps $\bar{G}_\ell$ themselves can also become control variables, learned or adapted from observed marginal value, instability, gradient disagreement, or validation response.

This suggests a two-timescale controller: fast per-mini-batch decisions select K_t , while a slower controller learns layer-specific starvation budgets $\bar{G}_\ell$. The resulting problem jointly learns the model and allocates the computational resource used to train it.

23 Discussion

23.1 What is canonical about exact backpropagation?

Exact reverse AD remains mathematically distinguished. It is the exact reverse derivative when the required dual structure exists, it has zero composition defect, and it gives the true gradient. The generalisation proposed here does not weaken those facts. It changes the optimisation question: the exact pullback is canonical as a semantic object, whereas the best learning procedure depends on what resources and outcomes are priced.

23.2 Why shortcutting and surrogate derivatives can fail

The experiments do not say that arbitrary feedback works. The straight-through rule and direct feedback are substantially weaker on deep logical extrapolation. Theorem 13.1 gives only a local sufficient condition and does not guarantee good representation learning, stability, or generalisation. Likewise, a small composition defect does not automatically imply a useful learning trajectory.

23.3 Why local semantic learning is strong here

Local semantic supervision has privileged information: every subformula has an exact target. That is available in propositional logic but not in most natural supervised-learning problems. Its role is methodological. It demonstrates that a procedure with zero inter-level credit transport and modest gradient alignment can nevertheless learn highly systematic shared operators when local targets are semantically correct. In less structured domains, synthetic or inferred local targets would be needed.

23.4 Relation to biological plausibility

The separation between primal and feedback graphs is closely related to concerns about weight transport and symmetric feedback [Lillicrap et al., 2016, Akroun et al., 2019]. Our framework is not a biological model, but it provides common mathematical coordinates—transport symmetry, feedback depth, locality, and signal fidelity—on which biologically motivated and engineering-motivated procedures can be compared.

23.5 Relation to numerical analysis

Once backward propagation is written as $(I - \mathcal{B})s = q$, several numerical ideas become available: Jacobi and Gauss–Seidel schedules, relaxation, preconditioning, multigrid-like coarse credit, Krylov subspace approximations, residual-based stopping, and asynchronous solvers [Saad, 2003]. For DAGs the exact system is finite and triangular, but approximate or distributed solvers can still be preferable under latency or communication constraints. For implicit networks the numerical analogy becomes literal.

23.6 What the MNIST experiment changes

The MNIST experiment is not merely an external replication of the propositional result. It falsifies the naive statement that a very short fixed horizon is generally sufficient. Parameter sharing across structural depth is the missing variable. In the AST experiment, AND, OR, NOT, and implication modules recur at many syntactic levels and can receive longitudinal credit through different formula occurrences. In LeNet, C_1 is always the earliest block. A fixed $K < 5$ never moves it closer to the loss. Thus propagation theory needs both a graph-theoretic horizon and a data/trajectory-dependent notion of parameter credit accessibility.

23.7 Fidelity, coverage, and cost as independent axes

The initial LeNet diagnostics show that a propagation rule may be globally close to the exact gradient while structurally incomplete. Conversely, DFA and local rules can have low global cosine yet contact every block. We therefore refine the propagation optimality vector to include at least

$$\mathcal{O}(P) = (R_{\text{final}}, T_{\varepsilon}, C_{\text{work}}, A_{\text{global}}, A_{\text{bal}}, (C_{\ell})_{\ell}, (G_{\ell})_{\ell}, M, \Lambda, L_{\text{local}}, R_{\text{robust}}), \quad (23.1)$$

where M is memory, Λ is latency, and L_{local} measures locality or communication. Exact reverse AD is optimal in a semantic-fidelity coordinate; it need not be Pareto-optimal once all coordinates are priced.

23.8 Statistical interpretation of the new image results

The MNIST study uses only three seeds and three epochs. It therefore establishes algorithmic phenomena, not definitive ranking. In particular, the difference between exact BP and local propagation after three epochs is small relative to the scale at which a larger replication would be required for a superiority claim. The adaptive rule likewise essentially ties $K = 4$ in short-horizon accuracy while using less structural reverse work; the present Python implementation does not yet turn that structural saving into a proportionate end-to-end wall-clock gain. The kernel microbenchmark indicates where such a gain could emerge in a fused or accelerator implementation.

24 Limitations and future work

The experiments remain deliberately interpretable and relatively small. Important limitations include:

1. only three seeds in the AST timing, credit-horizon, MNIST, and adaptive-image experiments;
2. short three-epoch MNIST runs designed for speed comparisons rather than converged benchmark performance;
3. CPU/Python wall-clock measurements rather than fused GPU or TPU kernels;
4. a structural work proxy based on traversed suffix blocks, which ignores heterogeneous layer costs;
5. small logical vocabularies and formula depths;
6. local semantic supervision in the logic experiment, which exploits exact subformula labels unavailable in most natural tasks;
7. no direct memory-bandwidth, distributed-communication, energy, or accelerator-utilisation measurements; and
8. no large-scale natural-image or language benchmark yet.

The most immediate empirical extensions are: (i) larger seed studies; (ii) full-convergence MNIST runs and larger CNNs; (iii) layer-cost-weighted rather than block-count credit budgets; (iv) learned coverage gaps $\tilde{G}_\ell$; (v) asynchronous and Gauss–Seidel-like message schedules; (vi) learned transport maps and synthetic gradients in the same AST and image models; and (vii) GPU experiments where feedback critical path, activation storage, and communication can dominate arithmetic.

At the theoretical level, a central problem is to relate operator-level propagation defect to multi-step optimisation trajectories while accounting simultaneously for alignment, parameter sharing, data-distribution coverage, and starvation gaps. Another direction is categorical: define a category or bicategory whose morphisms carry both primal maps and possibly inexact feedback maps, with reverse derivative categories embedded as a zero-defect substructure and coverage/cost added as quantitative enrichment.

25 Conclusion

The exact semantic development above establishes the result that reverse-mode automatic differentiation is adjoint pullback through the compositional structure of a differentiable program. The generalized theory preserves that result and places it inside a broader algorithmic theory. A backward learning procedure is described by its feedback graph, signal spaces, transports, aggregation, update interpreter, and numerical schedule. In the linear-additive case its signal field satisfies

$$s = q + \mathcal{B}s.$$

On a DAG, exact backpropagation is the finite Neumann solution and reverse traversal is the exact triangular solve. But learning does not require the exact solution on every update: positive alignment is sufficient for local descent, and the empirical studies show that useful learning can survive substantial departure from the exact gradient.

The propositional experiments establish three phenomena in a controlled setting: low-fidelity propagation rules can learn exact semantics; the speed-optimal or systematicity-optimal procedure need not be exact BP; and the empirically optimal credit horizon can be strictly shorter than graph depth. The MNIST experiments then identify the boundary of that conclusion. Fixed short horizons fail when they permanently starve early sequential layers, whereas local and random-feedback procedures can learn despite low gradient fidelity because they maintain broad credit access. This leads to temporal credit coverage and starvation gap as additional theoretical coordinates.

The coverage-constrained adaptive experiment demonstrates a concrete synthesis. Full reverse propagation is used only intermittently; the mean suffix horizon is about 3.56 of 5 blocks, yet three-epoch MNIST accuracy remains close to fixed $K = 4$ and exact BP. The resulting principle is more precise than either “backpropagation is necessary” or “local credit is enough”:

Optimal learning should minimise the cost of backward information transport subject to sufficient directional quality and sufficient credit coverage over time.

This turns backpropagation depth from a fixed architectural fact into a schedulable computational resource and suggests a new class of algorithms that jointly learn parameters and allocate reverse computation.

A Additional proof details

A.1 Relative-error descent bound

Write $d = g + e$ with $\|e\| \leq \rho \|g\|$. Then

$$\langle g, d \rangle = \|g\|^2 + \langle g, e \rangle \geq (1 - \rho) \|g\|^2,$$

and

$$\|d\| \leq \|g\| + \|e\| \leq (1 + \rho) \|g\|.$$

Substitution in the descent lemma yields the relative-error step-size bound stated in the main text.

A.2 Exactness of finite Jacobi propagation

By induction on K , the Jacobi recurrence gives

$$s^{(K)} = \sum_{j=0}^{K-1} \mathcal{B}^j q.$$

The nilpotence theorem then gives $s^{(D+1)} = s^*$. Thus parallel synchronous propagation can recover the exact adjoint field without reverse substitution, at the price of $D + 1$ synchronisation rounds.

A.3 Coverage-deadline optimality

The proof of Proposition 20.2 follows from suffix structure. If block ℓ is due and lies r_ℓ reverse blocks from the output, every feasible suffix horizon must satisfy $K_t \geq r_\ell$. Therefore every feasible choice satisfies

$$K_t \geq \max\{K_{\text{base}}, \max_{\ell \in D_t} r_\ell\},$$

and choosing equality is pointwise minimal among suffix policies satisfying the current deadlines.

B Reproducibility, Code, Data, and Figure Manifest

All executable references are collected in this appendix. The reproducibility archive contains the *separate runnable source files used for the experiments and diagnostics*, rather than a generated monolithic wrapper. Files that depend on another experiment module are kept in the same `code/` directory so their imports work without installation as a package. Exact commands and dependency installation are documented in `code/README.md`.

B.1 Runnable code modules

Table 8: Runnable source files supplied with the paper.

Path	Purpose
<code>code/backprop_flow_logic.py</code>	Fixed propositional truth-vector propagation experiment.
<code>code/summarize_truth_results.py</code>	Aggregates the five-seed truth-vector run into the summary used by Table 1.
<code>code/backprop_flow_propositional_ast_speed.py</code>	Compositional propositional-AST learning, timing, theoremhood, entailment, rewrite, and depth-generalisation experiment.
<code>code/backprop_credit_horizon_sweep.py</code>	Credit-horizon K sweep. It imports the AST experiment module from the same directory.
<code>code/credit_horizon_postprocess.py</code>	Produces the final K comparison and K^* optimum tables from the horizon summary.
<code>code/fetch_mnist.py</code>	Downloads the CSV-form MNIST train/test files expected by the LeNet experiments.
<code>code/mnist_backprop_lenet_experiment.py</code>	Three-seed MNIST/LeNet benchmark for exact BP, fixed horizons, DFA, and local propagation.
<code>code/mnist_gradient_diagnostics.py</code>	Reproduces global gradient cosine, layer-balanced alignment, and active-layer credit coverage diagnostics.
<code>code/adaptive_sparse_mnist_experiment.py</code>	Coverage-constrained adaptive sparse MNIST propagator; imports the LeNet module from the same directory.
<code>code/aggregate_adaptive_mnist.py</code>	Merges baseline and adaptive summaries into the comparison table used in the adaptive section.
<code>code/mnist_backward_kernel_microbenchmark.py</code>	Reproduces the 200-mini-batch forward/backward kernel timing table.
<code>code/make_paper_figures.py</code>	Regenerates the empirical paper figures from the experiment CSV artifacts.

B.2 Experiment-to-artifact map

Table 9: Experiment, source, data, and paper-output map.

Experiment	Primary runnable source	Primary data artifacts	Paper outputs
Fixed truth vectors	<code>code/backprop_flow_logic.py</code>	<code>data/backprop_flow_summary_5seeds.csv</code>	Table 1
AST logic and speed	<code>code/backprop_flow_propositional_ast_speed.py</code>	<code>data/backprop_ast_speed_3seeds_summary.csv</code>	Table 2; Figures 1–2
Credit-horizon sweep	<code>code/backprop_credit_horizon_sweep.py</code>	<code>data/backprop_credit_horizon_final_table.csv</code> ; <code>data/backprop_credit_horizon_frontier.csv</code> ; <code>data/backprop_credit_horizon_optima.csv</code>	Table 3; Figures 3–4

Experiment		Primary runnable source	Primary data artifacts	Paper outputs
MNIST/LeNet		<code>code/mnist_backprop_lenet_experiment.py</code> ; <code>code/mnist_gradient_diagnostics.py</code>	<code>data/mnist_backprop_summary_3seeds.csv</code> ; <code>data/mnist_initial_gradient_alignment.csv</code> ; <code>data/mnist_layerwise_gradient_alignment.csv</code>	Tables 4–5; Figures 5–7
Adaptive sparse MNIST		<code>code/adaptive_sparse_mnist_experiment.py</code> ; <code>code/mnist_backward_kernel_microbenchmark.py</code>	<code>data/adaptive_sparse_summary_3seeds.csv</code> ; <code>data/adaptive_vs_baselines.csv</code> ; <code>data/backward_kernel_microbenchmark.csv</code>	Tables 6, 7; Figures 8–9
Figure regeneration		<code>code/make_paper_figures.py</code>	The CSV artifacts above	All empirical figures in this appendix manifest

B.3 Figure files

All rendered figures used by the empirical sections are stored together under `figures/`:

- `figures/gradient_alignment_vs_depth6.png`;
- `figures/training_time_vs_depth6.png`;
- `figures/credit_horizon_accuracy.png`;
- `figures/credit_horizon_work_frontier.png`;
- `figures/mnist_speed_accuracy_frontier.png`;
- `figures/mnist_time_learning_curves.png`;
- `figures/mnist_propagation_diagnostics.png`;
- `figures/adaptive_structural_frontier.png`; and
- `figures/adaptive_horizon_usage.png`.

B.4 Timing interpretation

Reported wall-clock times are hardware- and implementation-dependent. The experiments separate core optimisation time from evaluation where possible. The adaptive study additionally reports a forward/backward-kernel microbenchmark. Structural work counts are intended as architecture-level proxies and should not be read as exact accelerator FLOP counts.

References

- Martin Abadi and Gordon D. Plotkin. A simple differentiable programming language. *Proceedings of the ACM on Programming Languages*, 4(POPL):38:1–38:28, 2020. doi: 10.1145/3371106.
- Samson Abramsky and Achim Jung. Domain theory. In Samson Abramsky, Dov M. Gabbay, and T. S. E. Maibaum, editors, *Handbook of Logic in Computer Science, Volume 3: Semantic Structures*, pages 1–168. Oxford University Press, 1994.
- Mohamed Akrouf, Collin Wilson, Peter C. Humphreys, Timothy Lillicrap, and Douglas Tweed. Deep learning without weight transport. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

- Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.
- Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- R. F. Blute, J. R. B. Cockett, and R. A. G. Seely. Differential categories. *Mathematical Structures in Computer Science*, 16(6):1049–1083, 2006. doi: 10.1017/S0960129506005676.
- R. F. Blute, J. R. B. Cockett, and R. A. G. Seely. Cartesian differential categories. *Theory and Applications of Categories*, 22(23):622–672, 2009. doi: 10.70930/tac/y9rglypb.
- R. F. Blute, J. R. B. Cockett, Jean-Simon Pacaud Lemay, and R. A. G. Seely. Differential categories revisited. *Applied Categorical Structures*, 28:171–235, 2020. doi: 10.1007/s10485-019-09572-y.
- J. R. B. Cockett and G. S. H. Cruttwell. Differential structure, tangent structure, and SDG. *Applied Categorical Structures*, 22:331–417, 2014. doi: 10.1007/s10485-013-9312-0.
- Robin Cockett, Geoffrey Cruttwell, Jonathan Gallagher, Jean-Simon Pacaud Lemay, Benjamin MacAdam, Gordon Plotkin, and Dorette Pronk. Reverse derivative categories. In *28th EACSL Annual Conference on Computer Science Logic (CSL 2020)*, volume 152 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 18:1–18:16. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2020. doi: 10.4230/LIPIcs.CSL.2020.18.
- Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2 edition, 2008. doi: 10.1137/1.9780898717761.
- Max Jaderberg, Wojciech Marian Czarnecki, Simon Osindero, Oriol Vinyals, Alex Graves, David Silver, and Koray Kavukcuoglu. Decoupled neural interfaces using synthetic gradients. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1627–1635. PMLR, 2017.
- Hoyong Kim and Kangil Kim. Spherization layer: Representation using only angles. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- Timothy P. Lillicrap, Daniel Cownden, Douglas B. Tweed, and Colin J. Akerman. Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7:13276, 2016. doi: 10.1038/ncomms13276.
- Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2 edition, 2006. doi: 10.1007/978-0-387-40065-5.
- Arild Nøkland. Direct feedback alignment provides learning in deep neural networks. *arXiv preprint arXiv:1609.01596*, 2016.

- Barak A. Pearlmutter. Fast exact multiplication by the hessian. *Neural Computation*, 6(1): 147–160, 1994. doi: 10.1162/neco.1994.6.1.147.
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.
- Yousef Saad. *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics, 2 edition, 2003. doi: 10.1137/1.9780898718003.
- Benjamin Scellier and Yoshua Bengio. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in Computational Neuroscience*, 11:24, 2017. doi: 10.3389/fncom.2017.00024.
- Penghang Yin, Jiancheng Lyu, Shuai Zhang, Stanley Osher, Yingyong Qi, and Jack Xin. Understanding straight-through estimator in training activation quantized neural nets. In *7th International Conference on Learning Representations (ICLR)*, 2019.